Core Concepts

Numbers to Know

Learn about the numbers you need to know for system design interviews.

Our industry moves fast. The hardware we build systems on evolves constantly, which means even recent textbooks can become outdated quickly. A book published just a few years ago might be teaching patterns that still make sense, but quoting numbers that are off by orders of magnitude.

One of the biggest giveaways that a candidate has book knowledge but no hands-on experience during a system design interview is when they rely on outdated hardware constraints. They do scale calculations using numbers from 2015 (or even 2020!) that dramatically underestimate what modern systems can handle. You'll hear concerns about database sizes, memory limits, and storage costs that made sense then, but would lead to significantly over-engineered systems today.

This isn't the candidate's fault – they're doing the right thing by studying. But understanding modern hardware capabilities is crucial for making good system design decisions. When to shard a database, whether to cache aggressively, how to handle large objects – these choices all depend on having an accurate sense of what today's hardware can handle.

Let's look at the numbers that actually matter in 2026.

Modern Hardware Limits

Modern servers pack serious computing power. An AWS **M6i.32xlarge** comes with 512 GiB of memory and 128 vCPUs for general workloads. Memory-optimized instances go further: the **X1e.32xlarge** provides 4 TB of RAM, while the **U-24tb1.metal** reaches 24 TB of RAM. This shift matters because many applications that once required distributed systems can now run on a single machine.

Storage capacity has seen similar growth. Modern instances like AWS's **i3en.24xlarge** provide 60 TB of local SSD (solid-state drive) storage. If you need more, the **D3en.12xlarge** offers 336 TB of HDD (hard disk drive) storage for data-heavy workloads. Object storage like **S3** is effectively unlimited, handling petabyte-scale deployments as a standard practice. The days of storage being a primary constraint are largely behind us.

Network capabilities haven't stagnated either. Within a datacenter, 25 Gbps is common for standard instances, with high-performance instances supporting 50-100 Gbps or more. Bandwidth across availability zones (AZs) within a region is limited only by instance network capacity. Latency remains predictable: sub-1ms within a single AZ, 1-2ms across AZs in the same region, and 50-150ms cross-region. This consistent performance allows for reliable distributed system design.

These aren't just incremental improvements – they represent a step change in what's possible. When textbooks talk about splitting databases at 100GB or avoiding large objects in memory, they're working from outdated constraints. The hardware running our systems today would have been unimaginable a decade ago, and these capabilities fundamentally change how we approach system design.

Applying These Numbers in System Design Interviews

Let's look at how these numbers impact specific components and the decisions we make when designing systems in an interview.

Caching

In-memory caches have grown exponentially in both size and capability. Gone are the days of 32-64GB Redis instances that required careful memory management and partial dataset caching. Today's caches routinely handle terabyte-scale datasets with single-digit millisecond latency, and a single instance can process hundreds of thousands of operations per second. This shift in scale changes the entire approach to caching strategy.

Numbers to know:

- Memory: Up to 1TB on memory-optimized instances, with some configurations exceeding this for specialized use cases
- Latency
  - Reads: < 1ms within the same region
  - Writes: < 1ms same-AZ, 1-2ms cross-AZ (within the same region) for optimized systems
- Throughput: 100k-200k+ operations/second per instance for in-memory caches like ElastiCache Redis on modern Graviton-based nodes (reads and writes are roughly equivalent for simple operations)

When to consider scaling:

- Dataset Size: Approaching 1TB in size
- Throughput: Sustained throughput of 100k+ ops/second
- Read Latency: Requirements below 0.5ms consistently

These capabilities fundamentally change caching strategy. The ability to cache entire databases in memory, even at hundreds of gigabytes, means you can often avoid complex partial caching schemes altogether. This "cache everything" approach, while seemingly brute force, typically costs less than engineering time spent on selective caching logic. When you do need to scale, the bottleneck is usually operations per second or network bandwidth, not memory size – a counterintuitive shift from just a few years ago.

Databases

The raw power of modern databases surprises even experienced engineers. Single PostgreSQL or MySQL instances now routinely handle dozens of terabytes of data while maintaining millisecond-level response times. This isn't just about storage either. Modern databases efficiently handle tens of thousands of transactions per second on a single primary, with the bottleneck often being operational concerns rather than performance limits.

Numbers to know:

- Storage: Single instances handle up to 64 TiB for most database engines, with Aurora supporting up to 256 TiB
- Latency
  - Reads: 1-5ms for cached data, 5-30ms for disk (optimized configurations for RDS and Aurora)
  - Writes: 5-15ms for commit latency (for single-node, high-performance setups)
- Throughput
  - Reads: Up to 50k TPS (transactions per second) in single-node configurations on Aurora and RDS
  - Writes: 10-20k TPS in single-node configurations on Aurora and RDS
- Connections: 5-20k concurrent connections, depending on database and instance type

When to consider sharding:

- Dataset Size: Approaching or exceeding 50 TiB may require sharding or distributed solutions
- Write Throughput: Consistently exceeding 10k TPS indicates scaling considerations
- Read Latency: Requirements below 5ms for uncached data may necessitate optimization
- Geographic Distribution: Cross-region replication or distribution needs
- Backup/Recovery: Backup windows that stretch into hours or become operationally impractical

While the largest systems in the world (social networks, e-commerce giants, etc.) absolutely need sharding to handle their scale, many candidates jump to distributed solutions too early. For systems handling millions or even tens of millions of users, a well-tuned single database can often handle the load. When you do need to scale, carefully consider what's driving the decision: is it pure data volume, operational concerns like backup windows, or the need for geographic distribution? Understanding these tradeoffs leads to better scaling decisions.

> A "single instance" doesn't mean a single point of failure. In practice, you'd still run a primary with read replicas for availability (e.g. Aurora's multi-AZ failover). The point here is that you often don't need to *shard* the data — replication for HA is a separate concern from horizontal partitioning for scale.

> More often than not I see candidates reaching for scaling too quickly. They have 500GB or a couple of terabytes of data and they're start explaining how they'd shard the database. Slow down, do the math, and make sure sharding is actually needed before you start explaining how you'd do it.

Application Servers

Modern application servers have evolved beyond the resource constraints that shaped many traditional design patterns. Today's servers routinely handle thousands of concurrent connections with modest resource usage, while cloud platforms enable rapid scaling in response to load. CPU processing power, rather than memory or connection limits, typically determines your server's capabilities.

Numbers to know:

- Connections: 100k+ concurrent connections per instance for optimized configurations
- CPU: 8-64 cores
- Memory: 64-512GB standard, up to 2TB available for high-memory instances
- Network: 25 Gbps standard, up to 50-100 Gbps on high-performance instances
- Startup Time: 30-60 seconds for containerized apps

When to consider horizontal scaling:

- CPU Utilization: Consistently above 70-80%
- Response Latency: Exceeding SLA or critical thresholds
- Memory Usage: Trending above 70-80%
- Network Bandwidth: Approaching instance limits

The implications for system design are significant. While the trend toward stateless services is valuable for scaling, don't forget that each server has substantial memory available. Local caching, in-memory computations, and session handling can all leverage this memory to improve performance dramatically. CPU is almost always your first bottleneck, not memory, so don't shy away from memory-intensive optimizations when they make sense. When you do need to scale, cloud platforms can spin up new instances in 30-60 seconds for containerized apps, making aggressive auto-scaling a viable alternative to over-provisioning. This combination of powerful individual instances and rapid scaling means you can often achieve high performance through simple architectures.

Message Queues

Message queues have transformed from simple task delegation systems into high-performance data highways. Modern systems like Kafka process **millions of messages per second** with single-digit millisecond latency, while maintaining weeks or months of data. This combination of speed and durability has expanded their role far beyond traditional async processing.

Numbers to know:

- Throughput: Up to 1 million messages/second per broker in modern configurations
- Latency: 1-5ms end-to-end within a region for optimized setups

- Message Size: 1KB-10MB efficiently handled
- Storage: Up to 50TB per broker in advanced configurations
- Retention: Weeks to months of data, depending on disk capacity and configuration

When to consider scaling:

- Throughput: Nearing 800k messages/second per broker
- Partition Count: Approaching 200k per cluster
- Consumer Lag: Consistently growing, impacting real-time processing
- Cross-Region Replication: If geographic redundancy is required

The performance characteristics of modern queues challenge traditional system design assumptions. With consistent sub-5ms latencies, you can now use queues within synchronous request flows—getting the benefits of reliable delivery and decoupling without forcing APIs to be async. This speed, combined with practically unlimited storage, means queues can serve as the backbone for event sourcing, real-time analytics, and data integration patterns that previously required specialized systems.

Cheat Sheet

Here is a one-stop-shop for the numbers you need to know in 2026. These numbers represent typical values for well-tuned systems with specific workloads - your requirements may vary based on workload, hardware, and configuration. Use them as a starting point for capacity planning and system design discussions, not as hard limits. Remember that cloud providers regularly update their offerings, so while I'll try to keep this up to date, it should be treated more as a starting point than a hard limit.

Component	Key Metrics	Scale Triggers
Caching	- ~1 millisecond latency - 100k+ operations/second - Memory-bound (up to 1TB)	- Hit rate < 80% - Latency > 1ms - Memory usage > 80% - Cache churn/thrashing
Databases	- Up to 50k transactions/second - Sub-5ms read latency (cached) - 64 TiB+ storage capacity	- Write throughput > 10k TPS - Read latency > 5ms uncached - Geographic distribution needs

Component	Key Metrics	Scale Triggers
App Servers	- 100k+ concurrent connections - 8-64 cores @ 2-4 GHz - 64-512GB RAM standard, up to 2TB	- CPU > 70% utilization - Response latency > SLA - Connections near 100k/instance - Memory > 80%
Message Queues	- Up to 1 million msgs/sec per broker - Sub-5ms end-to-end latency - Up to 50TB storage	- Throughput near 800k msgs/sec - Partition count ~200k per cluster - Growing consumer lag

Common Mistakes In Interviews

Premature sharding

The single biggest mistake I see candidates make is assuming **sharding** is always necessary. They introduce a data model and immediately explain which column they'd shard on. It comes up almost every time with **Design Yelp** in particular. Here we have 10M businesses, each of which is roughly 1KB of data. This is `10M * 1KB = 10GB` of data! 10x it to account for reviews which we can store in the same database and you're only at `100GB`, why would you shard?

The same thing comes up a lot with caches. Take a **LeetCode** leaderboard where we have 100k competitions and up to 100k users per competition. We're looking at `100k * 100k * (36B ID + 4B float rating) = 400GB`. While even more than what we store on disk with Yelp, this can still fit on a single large cache -- no need to **shard**!

Overestimating latency

I see this most with SSDs. Candidates tend to vastly overestimate the latency to query an SSD (Database) for a simple key or row lookup. We're talking sub-millisecond to a few milliseconds for indexed lookups. It's fast! Candidates will oftentimes justify adding a caching layer to reduce latency when the simple row lookup is already fast enough -- no need to add additional infrastructure.

> ⚠ Note, this is only for simple row lookups with an index. It is still wise to cache expensive

> queries.

Over-engineering given a high write throughput

Similar to the above, incorrect estimates routinely lead to over-engineering. Imagine we have a system with 5k writes per second. Candidates will often jump to adding a message queue to buffer this "high" write throughput. But they don't need to!

Let's put this in perspective. A well-tuned Postgres instance with simple writes can handle 20k+ writes per second. What actually limits write capacity are things like complex transactions spanning multiple tables, write amplification from excessive indexes, writes that trigger expensive cascading updates, or heavy concurrent reads competing with writes. If you're just inserting rows or doing simple updates with proper indexes, there's no need for complex queueing systems at 5k WPS.

Message queues become valuable when you need guaranteed delivery in case of downstream failures, event sourcing patterns, handling write spikes that exceed your database's capacity (e.g. above 20k+ WPS for a single Postgres instance), or decoupling producers from consumers. But they add complexity and should be justified by actual requirements. Before reaching for a message queue, consider simpler optimizations like batch writes, optimizing your schema and indexes, using connection pooling effectively, or using async commits for non-critical writes.

The core point is to understand your actual write patterns and requirements before adding infrastructure complexity. Modern databases are incredibly capable, and simple solutions often perform better than you might expect.

What about costs?

When you're actually designing a real system, cost is often a major factor. You'll sketch out estimates for your usage, multiply those by pricing tables, and try to compare the exact dollar costs of different options to establish a TCO (Total Cost of Ownership).

But in system design interviews, this is rarely the focus. First, most candidates and interviewers don't care to memorize pricing tables (and RAM or SSD prices are changing *constantly*). And secondly, accurate pricing requires good estimates, and remember for our SD estimates we're usually looking for an order of magnitude: not an exact dollar amount.

So interviewers tend to not be especially sensitive to exact costs. That's not to say you should ignore the abstract idea of whether something is cost-effective or not. Having 100 machines when 1 will do, or using a bank of in-memory caches when users only need their data in hundreds of milliseconds will trigger a flag from an interviewer. But we would not recommend you memorize AWS pricing tables — spend your time elsewhere.

Conclusion

Modern hardware capabilities have fundamentally changed the calculus of system design. While distributed systems and horizontal scaling remain necessary for the world's largest applications, many systems can be significantly simpler than what traditional wisdom suggests.

Understanding these numbers helps you make better scaling decisions:

- Single databases can handle terabytes of data
- Caches can hold entire datasets in memory
- Message queues are fast enough for synchronous flows (as long as there is no backlog!)
- Application servers have enough memory for significant local optimization

The key insight isn't that vertical scaling is always the answer – it's knowing where the real limits are. This knowledge helps you avoid premature optimization and build simpler systems that can grow with your needs. In system design interviews, demonstrating this understanding shows that you can balance theoretical knowledge with practical experience – a crucial skill, especially for the more senior levels.